arXiv:2411.02193v2 [cs.LG] 21 Nov 2024

IMPROVING STEERING VECTORS BY TARGETING SPARSE AUTOENCODER FEATURES

Sviatoslav Chalnev[*]
slava.chalnev@gmail.com

Matthew Siu[*]
matthewwilsonsiu@gmail.com

Arthur Conmy
arthurconmy@gmail.com

ABSTRACT

To control the behavior of language models, steering methods attempt to ensure that outputs of the model satisfy specific pre-defined properties. Adding steering vectors to the model is a promising method of model control that is easier than finetuning, and may be more robust than prompting.However, it can be difficult to anticipate the effects of steering vectors produced by methods such as CAA [Panickssery et al., 2024] or the direct use of SAE latents [Templeton et al., 2024]. In our work, we address this issue by using SAEs to measure the effects of steering vectors, giving us a method that can be used to understand the causal effect of any steering vector intervention. We use this method for measuring causal effects to develop an improved steering method, **SAE-Targeted Steering** (SAE-TS), which finds steering vectors to target specific SAE features while minimizing unintended side effects. We show that overall, SAE-TS balances steering effects with coherence better than CAA and SAE feature steering, when evaluated on a range of tasks.

1 Introduction

There are widespread calls for better control of the behaviour of Large Language Models (LLMs; e.g. The White House [2023]). Current methods such as prompting [Wallace et al., 2024] and finetuning [Ouyang et al., 2022, Chung et al., 2022] offer some degree of control, but have clear limitations. For example, prompting can be fragile and is often susceptible to methods that can subvert these instructions [Wei et al., 2023]. Finetuning a model can be more robust but requires a curated dataset for training which can be both expensive and time-consuming to produce (e.g. Dubey et al. [2024]).

Steering vectors [Turner et al., 2024] have the potential to be more robust than prompting, and both cheaper and easier to implement than finetuning. Steering vectors work by adding activations to the hidden state of a model, part way through the forward pass (Section 3). By generating and inserting activation vectors into the model's forward pass, we can steer the model towards desired behaviors.

However, a problem with current steering methods is their unpredictability – it's often unclear exactly how a steering vector will affect model behavior. Steering vectors may not produce the intended changes in the model's output or may cause unforeseen behaviors, as we discuss in Section 3. In some cases, steering interventions produce no interpretable changes in model behavior other than model degradation, as we show in Section 5. This unpredictability makes it difficult to precisely control the model.

To address these challenges, we develop a method for quantifying the effects of a steering intervention on model outputs. We use Sparse Autoencoders (SAEs; Ng [2011], Cunningham et al. [2023], Bricken et al. [2023]) to measure the change in feature activations caused by steering interventions. This lets us understand what change in behavior to expect from any particular steering intervention. Note: our analysis is *not* about how early SAEs impact later layer SAEs (as studied

[*]Equal contribution
[†]Code available at `https://github.com/slavachalnev/SAE-TS`

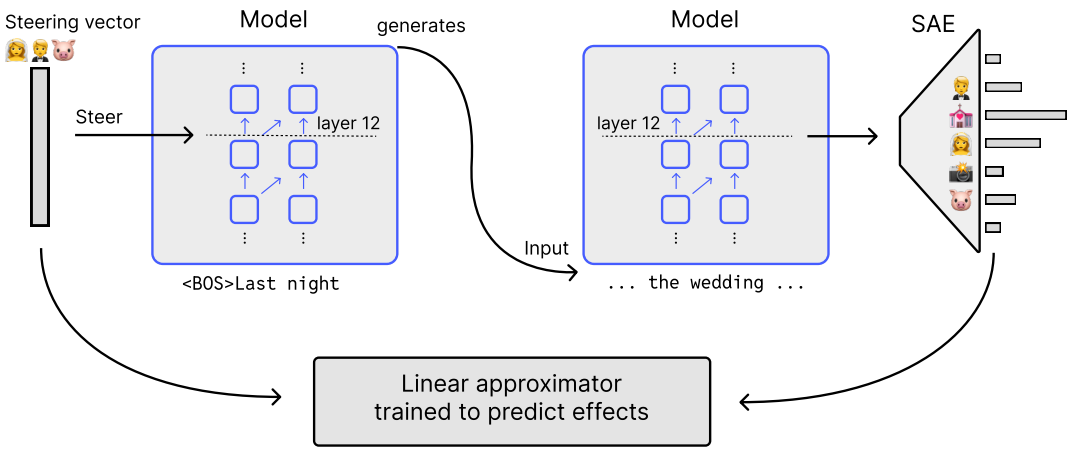

Figure 1: Diagram showing the feature effects measurement method. We sample a steering vector (depicted here as 🙆‍♀️👨‍🐷 to emphasise that it represents a mixture of concepts), and steer the model by adding it to the residual stream activations at layer 12. We use this steered model to generate text completions starting from a prompt (e.g. "<BOS>Last night"). These completions are then fed back through the model up to layer 12, where the activations are passed through a Sparse Autoencoder (SAE) and averaged to measure feature effects. The linear approximator is trained on pairs of steering vectors and their measured feature effects to predict effects for new steering vectors.

by prior work such as Marks et al. [2024] and Anders and Bloom [2024]). Instead, we look at how rollouts generated by a steered model differ from those generated by the unsteered, base model.

Building on this feature effects measurement method, we introduce **SAE-Targeted Steering** (SAE-TS), a method that constructs steering vectors to specifically target desired SAE features while minimizing unintended side effects. SAE-TS involves learning a linear relationship between steering vectors and their effects on SAE features. This allows us to construct steering vectors using the interpretable features found by the SAE. Notably, SAE-TS uses the SAE for steering differently to prior work such as Templeton et al. [2024] and Durmus et al. [2024]. We use the SAE to measure the effects of steering vectors, rather than directly adding SAE latents to model forward passes.

We evaluate SAE-TS against existing methods such as Contrastive Activation Addition (CAA) [Panickssery et al., 2024] and direct SAE feature steering Templeton et al. [2024]. Our evaluations demonstrate that SAE-TS outperforms existing methods, achieving better alignment with the intended behavior while maintaining the semantic coherence of the generated text across various tasks.

Key Contributions:

1. We develop a method to quantify and interpret the effects of a steering intervention using SAEs (Section 3).
2. We introduce SAE-Targeted Steering (SAE-TS), a method that constructs steering vectors to achieve specific desired effects while minimizing unintended changes (Section 4).
3. We evaluate SAE-TS on a set of steering tasks (Section 5), showing that it outperforms existing methods, successfully steering the model while maintaining output quality.

2 Related work

Activation Steering is a method for controlling model outputs introduced in Turner et al. [2024] where steering vectors are extracted by taking the difference between activations from pairs of contrasting positive and negative prompts. Panickssery et al. [2024] scaled this method to Llama-2, and Zou et al. [2023] use the methodology on a somewhat wider range of tasks. Recently, Cao et al. [2024] offers a way to learn a steering vector to optimally steer a model toward producing desired outputs. As with the activation steering methods above, a dataset of positive and negative examples is required for this method to work. Lee et al. [2024] introduce conditional steering. Scherlis et al. [2024] also studies the causal limitations of existing steering methods.

Mechanistic Interpretability and SAEs. Mechanistic Intepretability aims to understand how LLMs function internally by breaking down the models into understandable components [Olah et al., 2020, Olah]. The recent development of **sparse autoencoders** [Ng, 2011, Bricken et al., 2023, Cunningham et al., 2023] gives us a way to decompose the residual stream of transformer models into sparse and often human understandable features. Sparse autoencoders

provide evidence supporting the hypothesis that the latent space of LLMs is composed of linear and interpretable directions [Arora et al., 2018, Elhage et al., 2022]. Templeton et al. [2024] and Zhao et al. [2024] demonstrate that these directions found by the SAEs can be used for steering.

3 Measuring Steering Effects

As discussed in the introduction (Section 1), it can be difficult to predict the behavior of steering vectors. Measuring the effects of a steering intervention is useful for both interpreting the causal role of features and designing more effective steering vectors.

In transformer models, we can represent the forward pass as $h_n = b_n(h_{n-1})$, where b_i denotes the ith block and h_i is the hidden state after layer i. When we insert a steering vector $\boldsymbol{v}$ at a specific layer l, we modify the hidden state as $h_l \leftarrow h_l + \alpha \boldsymbol{v}$, where α is a scaling coefficient that determines the norm of the added steering vector.

We use JumpReLU SAEs [Rajamanoharan et al., 2024], which consist of an encoder f and a decoder, where the reconstruction $\hat{\boldsymbol{x}}$ of the input $\boldsymbol{x}$ is given by:

$$\hat{\boldsymbol{x}} = f(\boldsymbol{x})\boldsymbol{W}_{\text{dec}} + \boldsymbol{b}_{\text{dec}}, \tag{1}$$

and the SAE is trained such that $\hat{\boldsymbol{x}}$ approximates $\boldsymbol{x}$. SAE steering involves using the decoder vector d_i of feature i, which is the ith row of $\boldsymbol{W}_{\text{dec}}$, as the steering vector.

In order to measure a steering intervention's effect on the model output, we look at the difference in SAE feature activations between steered model outputs and original, unsteered model outputs. Our method is outlined at a high level in Figure 1 and can be summarised as follows:

1. **Data Generation**: Generate text rollout datasets D_{steered} and $D_{\text{unsteered}}$ using steered and unsteered models respectively, where the steered model has a steering vector $\boldsymbol{v}$ inserted at layer l.
2. **Feature Extraction**: For each generated output, pass the text back through the model up to layer l, and use the SAE encoder f to extract the feature activations at each position.
3. **Effect Computation**: Compute the average feature activations across all outputs and positions for both the unsteered and steered models, and calculate the difference to obtain the *steering effects* vector $\boldsymbol{y} \in \mathbb{R}^{d_{\text{SAE}}}$:

$$\boldsymbol{y} = \mathbb{E}_{\text{steered}}[f(\boldsymbol{x})] - \mathbb{E}_{\text{unsteered}}[f(\boldsymbol{x})]. \tag{2}$$

We use the Gemma-2-2B model [Gemma Team, 2024] and a Gemma Scope layer 12 residual stream SAE with hidden dimension of 16,384 [Lieberum et al., 2024]. We run the above procedure for every feature of the SAE, generating 896 rollouts of length 32, and computing the average difference between the steered and unsteered feature activations.

Table 1 shows a cherry-picked example of a feature which, when used for steering, has feature effects that are not itself. This feature (id 12904) activates on months (e.g. "January") but steering with its decoder vector causes the model to output digits of years. Upon closer inspection we see that the feature only activates on months following a day (e.g. 1st of January), so it is unsurprising that steering with this feature causes the model to output years. Some of the largest feature effects are features which are not clearly interpretable but are instead commonly occurring. E.g. in Table 1 the top effect is feature id 6810 with an activation difference more than 3 times higher than the next highest activation difference.

Given our initial motivation to use steering effects to develop better causal explanations of a feature's *role* in the model, features such as the month feature, which only have significant effects on features other than themselves, are of particular interest. When inspecting these features, we find that the effects encode many reasonable grammatical and semantic relationships. Another example is a feature that activates on "say", but increases a feature that activates on "hello" (see Appendix H).

4 Targeted Steering

We can use the above method of measuring steering effects to find steering vectors which steer the model towards a desired feature effect, while having minimal side-effects.

We do this by first training a linear **effect approximator** function (see Section 4.1) to predict the feature effects for any given steering vector and then use this function to find targeted steering vectors to achieve a desired effect.